

長庚大學資訊管理學系

碩士論文

Department of Information Management

Chang Gung University

Master Thesis

混合演算法求解多維度背包問題之研究

The Study of Hybrid Algorithms for Solving the

Multidimensional Knapsack Problem

研究生：紀勻翔

Graduate Student： Yun-Xiang Chi

指導教授：林詩偉 博士

Advisor： Shih-Wei Lin, Ph.D.

中華民國 113 年 7 月

July 2024

中文摘要

本研究旨在改進多維度背包問題(Multi-dimensional Knapsack Problem, MKP)的解決方法。引入基於線性鬆弛的評估方法，從而在搜尋過程中更有效地更新編碼。此外提出混合黏菌和正弦餘弦演算法(Hybrid Slime Mould and Sine Cosine Algorithm, HSMSCA)，該演算法結合黏菌演算法(Slime Mould Algorithm, SMA)和正弦餘弦演算法(Sine Cosine Algorithm, SCA)，並加入了強化學習機制，優化 MKP 的求解策略。本研究透過在四種不同規模的題庫上進行實驗，驗證了演算法的有效性，實驗結果表明，相比於現有的其他元啟發式教法，本文提出的 HSMSCA 演算法表現出了較佳的效能。

關鍵詞：多維度背包問題、黏菌演算法、正弦餘弦演算法、線性鬆弛、全局搜索效能，離散型優化問題

英文摘要

This study aims to improve the solution methods for the multi-dimensional knapsack problem (MKP). It introduces an evaluation method based on linear relaxation, which effectively adjusts the positions of solutions during the search process. A modified hybrid algorithm is developed, integrating the Slime Mould Algorithm (SMA) and the Sine Cosine Algorithm (SCA) with a reinforcement learning mechanism to optimize the MKP solving strategy. The effectiveness of the algorithm was validated through experiments on four test libraries of varying scales. The experimental results indicate that the proposed algorithm outperforms existing meta-heuristic methods, demonstrating superior performance.

Keywords: Multi-dimensional Knapsack Problem, Slime Mould Algorithm, Sine Cosine Algorithm, linear relaxation, global search performance, discrete optimization problems.

目 錄

中文摘要.....	i
英文摘要.....	ii
目 錄.....	iii
圖目錄.....	vi
表目錄.....	vii
第一章 緒論.....	1
1.1 研究動機.....	1
1.2 研究目的.....	2
1.3 章節結構.....	3
1.4 研究流程.....	4
第二章 文獻探討	6
2.1 多維度背包問題	6
2.2 黏菌演算法	9
2.2.1 黏菌演算法基本原理	10
2.2.2 黏菌演算法流程	12
2.3 正弦餘弦演算法	13
2.3.1 正弦餘弦演算法基本原理	14

2.3.2 正弦餘弦演算法流程	17
第三章 研究方法	19
3.1 編碼表示方法	19
3.2 種群初始化方法	19
3.3 可行解生成方法	21
3.3.1 轉換函數.....	22
3.3.2 修復演算法	22
3.3.3 改進演算法	23
3.4 改進正弦餘弦演算法	25
3.5 混合黏菌和正弦餘弦演算法	25
3.6 改進黏菌演算法小型 MKP 問題數值示例.....	27
第四章 實驗結果	32
4.1 演算法開發的環境	32
4.2 題庫來源.....	32
4.3 比較對象與評估準則	34
4.4 參數實驗.....	35
4.5 比較結果.....	36
第五章 結論與未來研究方向	38
5.1 結論.....	38

5.2 未來研究方向	38
參考文獻.....	40
附錄.....	45

圖目錄

圖1 研究流程圖	5
圖2 SMA 演算法虛擬程式碼.....	13
圖3 不同 r_1 的值對解搜索區域的影響	16
圖4 正弦餘弦演算法的效果	17
圖5 SCA 演算法虛擬程式碼.....	18
圖6 初始精英種群生成的虛擬程式碼	21
圖7 修復演算法虛擬程式碼	23
圖8 改進演算法虛擬程式碼	24
圖9 ISMA 演算法虛擬程式碼	24
圖10 ISCA 演算法虛擬程式碼	25
圖11 HSMSCA 演算法流程圖	26
圖12 HSMSCA 演算法虛擬程式碼.....	27

表目錄

表1 SMA 演算法的概要.....	10
表2 SCA 演算法的概要.....	14
表3 小型 MKP 的資源消耗和利潤.....	27
表4 計算完性價比率後降冪排序	28
表5 題庫規模.....	33
表6 參數實驗結果	35
表7 比較與評估結果	37

第一章 緒論

多維度背包問題 [1, 2] 可以被視為背包問題(Knapsack Problem, KP) [3] 的拓展或延伸。這是一個經典的組合優化問題，在現實世界中，多維度背包問題(Multi-dimensional Knapsack Problem, MKP) 的模型被廣泛應用於許多領域。

1.1 研究動機

MKP 是一個在多個領域中具有重要影響力的優化問題，例如在集裝箱裝載 [4]、貨物裝載 [5]、物品選擇 [6]、資源配置 [7] 等方面。在這些應用中，MKP 的目標是選取一組物品子集，以滿足多種資源限制並且最大化整體利潤。

在實際應用中，MKP 相較於單一背包問題更為複雜，因為它牽涉到多個資源和多個資源限制。這導致問題的維度和複雜性增加，因此解決 MKP 變得更加困難。

對於 MKP 的解決方法，如動態規劃和分支界定等，其在處理大規模問題時的效率常不盡理想。因此，一些啟發式演算法應運而生。儘管這些演算法在解決 MKP 方面取得了一定進展，但仍需克服局部最優解陷阱，尤其是在處理大規模問題時。本研究旨在提出一系列改

進方法，以提升演算法效能，特別針對大規模問題進行優化。

1.2 研究目的

背包問題是一個廣泛應用於不同領域的組合優化問題，其中 MKP 在實際應用中扮演著重要的角色。正如前述，MKP 的複雜性源自多種資源和多個限制條件，這使得解決 MKP 變得極具挑戰性。

儘管許多最佳化演算法已被應用於 MKP，但在處理大規模問題時，許多演算法仍存在效能不足的問題。特別是在解決 MKP 時，陷入局部最佳解的情況是一個常見問題，這會限制演算法的全域搜索能力。以下是本研究的研究目的：

- (1) 提出採用線性鬆弛方法處理 MKP 的改進黏菌演算法(Improved Slime Mould Algorithm, ISMA)和改進正弦餘弦演算法(Improved Sine Cosine Algorithm, ISCA)。
- (2) 提出混合黏菌和正弦餘弦演算法(Hybrid Slime Mould and Sine Cosine Algorithm, HSMSCA)並引入強化學習機制進行了研究。
- (3) 透過常見的四種規模的題庫進行 MKP 問題的解決，研究結果顯示，相較於 ISMA、ISCA 演算法以及其他啟發式演算法，HSMSCA 呈現出更優異的效能。

1.3 章節結構

第二章的文獻回顧將專注於探討 MKP 相關的文獻和先前的研究成果，包括優化演算法和精確演算法。將詳細探討各種解決 MKP 的方法，特別關注它們的優勢和限制，以及 SMA 演算法和 SCA 演算法的基本運作原理。

第三章將介紹研究方法，包括 SMA 演算法、SCA 演算法以及其混合形式 MKP 求解方法。本章將討論演算法在 MKP 中的應用，包括轉換函數的選擇、可行解生成方法、ISMA 演算法、ISCA 演算法以及混合黏菌和正弦餘弦演算法(Hybrid Slime Mould and Sine Cosine Algorithm, HSMSCA)。最後，將提供小型 MKP 問題的數值示例。

第四章將深入探討演算法的開發與評估，內容包括開發環境、題庫來源、比較對象與評估準則、參數實驗及比較結果。首先介紹開發所使用的軟硬體環境，其次說明測試題庫的來源與構成。然後列出比較的現有演算法及評估準則，確保評價客觀公正。隨後詳細描述參數調優實驗，並確定最佳參數組合。最後呈現和分析實驗結果，通過數據對比不同演算法的表現，評價所開發演算法是否達到預期目標。

第五章總結本研究的成果並提出未來的研究方向。在結論部分，本研究運用了 SMA、SCA 和 HSMSCA 演算法的二元版本，並採用

線性鬆弛方法處理 MKP 問題。透過結合 SMA 和 SCA 形成新型混合演算法 HMSCA，並引入強化學習機制以提升編碼更新策略，從而提高解的尋找效率。研究結果顯示，HMSCA 在解決 MKP 方面優於原始 ISMA、ISCA 以及其他元啟發式演算法。未來的研究方向包括兩個主要部分：一是探索 HMSCA 在其他最佳化問題中的應用，例如多目標背包問題和機器學習特徵篩選；二是改進演算法，引入正交學習、深度強化學習和自適應學習策略等方法，以增強搜索能力。本章提供了後續研究的指引。

1.4 研究流程

透過文獻回顧以瞭解該領域現有研究的現況，從這些相關文獻中獲得研究目的和研究架構的概念，同時明確定義研究問題的範疇。接著，著手設計針對該問題的演算法，並進程式編寫和題庫測試。隨後，對實驗結果進行詳細的分析和闡釋。最終，結合實驗結果，提出本研究的結論，並探討未來可能的改進方向和更深入的研究建議。本研究之流程如圖1所示。

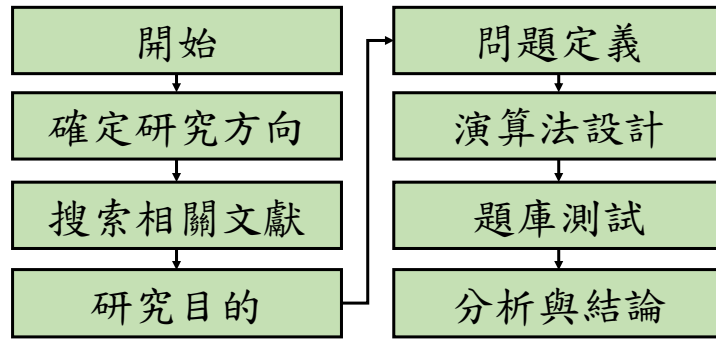


圖 1 研究流程圖

第二章 文獻探討

多維度背包問題(Multi-dimensional Knapsack Problem, MKP)是組合優化領域中一個經典挑戰，在眾多實際應用中扮演關鍵角色。本研究專注於探討使用黏菌演算法(Slime Mould Algorithm, SMA)和正弦餘弦演算法(Sine Cosine Algorithm, SCA)的新模型來解決 MKP 問題。SMA 演算法源自生物行為的啟發，而 SCA 演算法則基於數學特性的啟發，近年來在組合優化領域中備受矚目。本研究的文獻回顧將探討 MKP、SMA 演算法、SCA 演算法的相關研究，並介紹如何將 SMA 演算法和 SCA 演算法應用於解決 MKP 問題。

2.1 多維度背包問題

MKP 具體描述如下。給定一組物品 n ，每個物品都有利潤 p_i 和 m 個資源限制，每個資源皆具重量 w_{ji} 。表示第 i 個物品消耗的第 j 個資源的數量。同時有 m 個背包，每個背包都有容量 c_j 。MKP 的目標是獲得一組物品的子集，以最大化利潤同時滿足資源限制條件。MKP 的具體定義如下：

目標式：

$$\max F(X) = \sum_{i=1}^n p_i x_i \quad (1)$$

限制式：

$$\sum_{i=1}^n w_{ji} x_i \leq c_j, j = 0, 1, 2, \dots, m \quad (2)$$

$$x_i = 0 \text{ or } 1, i = 0, 1 \dots n \quad (3)$$

$$p_i > 0, w_{ji} > 0, c_j > 0 \quad (4)$$

若有任何違反容量限制的決策，將導致 x_j 為0；同樣地，任何違反後者的行為將導致第 i 限制被消除。

MKP 的解決方法可分為兩大類：精確演算法和啟發式演算法。精確演算法包括動態規劃 [8]、分支定界演算法 [9]、回溯演算法 [10]、枚舉演算法 [11]等。由於 MKP 屬於 NP 困難問題，這些方法僅適用於解決小規模問題，其計算時間可接受。然而，隨著 MKP 中物品數量和限制條件的增加，這些確定性方法顯示出計算效率不足的問題。

第二類別為啟發式演算法，在解決 MKP 方面受到較多關注。這些方法主要包括近似演算法 [12]、隨機演算法 [13]和元啟發式演算法。文獻中，各種元啟發式演算法如粒子群優化演算法 [14]、禁忌搜

索 [2]、鯨魚優化演算法 [15]、灰狼優化演算法 [16]。因此它們被視為克服使用精確技術解決 MKP 所需時間較長的有效方法，尤其是針對大規模問題。許多元啟發式算法已被應用於解決 MKP，其中一些算法將在本節其餘部分進行回顧。

Abdel-Basset et al. [17]中提出了一種新的改良多宇宙優化算法 (Modified Multi-Verse Optimization, MMVO)，專門用於解決背包問題和 MKP。MMVO 採用了一種兩階段修復策略來處理限制條件，並使用懲罰函數為不可行解賦予負值，以確保它們的適應值不會超過可行解的適應值。為了避免局部陷阱，MMVO 會在每次預定的迭代次數後重新初始化族群，同時保留到目前為止獲得的最佳解。此外 MMVO 採用 V 型轉移函數來離散化編碼。Abdel-Basset et al. [18]又提出了一種針對 MKP 的二元版本黏菌演算法(Binary Slime Mould Algorithm, BSMA)。由於 SMA 最初設計於連續優化問題，不適合直接應用於離散的 MKP，因此，發展了三種不同的適應函數(V 型、S 型、U 型)，並將其與標準演算法結合，形成了適合解決 MKP 的 BSMA。為了克服這一變體容易陷入局部陷阱的問題，作者還引入了兩種改進策略：隨機翻轉未選中物品的策略和迭代運算次數後的群體重新初始化策略。這些改進組成了一個更高效的演算法變體，稱為改進二元黏菌

演算法(Improved Binary Slime Mould Algorithm, IBSMA)。該研究還包括了一種修復機制來處理限制條件和不可行解，並通過與其他先進演算法的比較，證明了 IBSMA 在解決小型和大型 MKP 問題上的效能。

Feng et al. [19]提出了一種新的混合學習蛾搜尋算法(Hybrid Learning Moth Search, HLMS)，專門用於解決 MKP。HLMS 通過整合全局最佳和諧搜索(global-best harmony search, GHS)和 Baldwinian learning 兩種機制，來提高個體對價值空間的搜索效率和利用其他個體的訊息來改變搜索空間。GHS 主要負責全局探索，而 Baldwinian learning 則專注於局部開發。

2.2黏菌演算法

李世民等人 [20]於2020年提出的 SMA 演算法是一種新型的元啟發演算法，其主要模擬了自然界中黏菌覓食過程中的行為和形態變化。相較於其他智慧優化演算法，SMA 演算法具有原理簡單、調節參數少、尋優能力強、便於實現等優勢。

黏菌是一種生活在潮濕寒冷環境中的真核生物，是一種生活在潮濕且寒冷環境中的真核生物，其主要從外部有機物中獲取營養。當黏菌接近食物源時，生物震盪器將通過靜脈產生一個傳播波，以增加細

胞質流量。食物濃度越高，生物震盪器產生的傳播波也就越強，細胞質流動越快。SMA 演算法通過模擬黏菌的捕食行為來實現智慧尋優功能。

SMA 演算法技術已被廣泛應用於多個領域，包括太陽能光電電池參數估計 [21]、都市需水量預測 [22]、支持向量迴歸預測方法 [23] 等。通常會將 SMA 演算法與其他最佳化演算法結合使用，以提高效能。以下是一些近年改良 SMA 演算法的概要，詳見表1。

表1 SMA 演算法的概要

算法名稱	年份	描述
SMA-AGDE [24]	2021	一種高級混合方法，它結合使用 SMA 和自適應引導分差進化演算法來處理組合優化問題。
MSMA [25]	2022	一種多策略 SMA，它利用新穎的包圍食物技術和內積算法來優化水電多水庫系統。
mSMA [26]	2022	基於正交位置學習的 SMA 改進版最大功率點跟踪方法。
FP-SMA [27]	2022	增強型 SMA 使用自適應種群技術進行全局優化。
CO-SMA [28]	2022	用於最小化風力渦輪機能源成本的混沌對立改進 SMA。
ESMA [29]	2022	一種混合 SMA 和正餘弦算法進行全局優化的混合方法。
SMAFA [30]	2022	基於螢火蟲算法和 SMA 的混合方法進行特徵選擇。

2.2.1 黏菌演算法基本原理

在接近食物階段中，黏菌通過空氣中的氣味接近食物，用公式(5)表示其逼近行為， $\overrightarrow{vb}$ 是在 $[a, -a]$ 如公式(6)所示。

$$\vec{S}(t+1) = \begin{cases} X_b(t) + \overrightarrow{vb}(W \times \vec{S}_A(t) - \vec{S}_B(t)) & , r < p \\ \overrightarrow{vc} \times \vec{S}(t) & , r \geq p \end{cases} \quad (5)$$

$$\vec{vb} = [-a, a] \quad (6)$$

$$a = \operatorname{arctanh}\left(-\left(\frac{t}{t_{\max}}\right) + 1\right) \quad (7)$$

$\vec{vb}$ 為 $[-a, a]$ 之間的向量， $\vec{vc}$ 為1到0的線性遞減的向量， t 為當前迭代次數， $t_{\max}$ 為終止條件的最大迭代次數， r 是一個介於0和1之間的隨機值。 $\vec{S}_b$ 是當前已知最佳的個體位置， $\vec{S}(t+1)$ 為新生成的候選個體， $\vec{S}(t)$ 是個體的當前位置， $\vec{S}_A(t)$ 和 $\vec{S}_B(t)$ 從種群中隨機選擇兩個不重複個體。公式(5)中的 p 值計算如公式(8):

$$p = \tanh(|f(i) - DF|) \quad (8)$$

其中 $i \in 1, 2, 3, \dots, N$ ， $f(i)$ 是當前 $\vec{S}$ 的適應值， DF 是當前迭代中獲得的最佳適應值，即全局最佳適應值。 $\vec{W}$ 表示黏菌的權重，計算公式如下:

$$\vec{W}(\text{smellindex}(l)) = \begin{cases} 1 + r \times \log\left(\frac{bF - F(i)}{bF - wF} + 1\right), & \text{condition} \\ 1 - r \times \log\left(\frac{bF - F(i)}{bF - wF} + 1\right), & \text{other} \end{cases} \quad (9)$$

$$\text{smellindex} = \text{sort}(S) \quad (10)$$

其中 r 是一個介於0和1之間的隨機值， bF 表示當前迭代運算中的最佳適應值、 wF 表示當前迭代運算中的最差適應值， smellindex 表示排序後適應值的序列， condition 表示前50%種群[20]。 $F(i)$ 表示當前個體適應值。

在模擬黏菌的靜脈組織結構中的收縮模式時，可以透過數學步驟

來模擬其對食物的包圍行為。當考慮到食物品質時，黏菌會產生新的解或位置，其中食物濃度較高的區域將具有較大的權重，這將導致該區域的權重轉向探索其他區域。當食物濃度高時，黏菌需要決定何時離開當前區域前往另一區域，以尋找更多不同的食物來源而非僅限於當前最佳的食物來源。透過數學模型更新黏菌的位置，可以重新表達其在另一區域尋找多種食物來源的行為方式。

$$\vec{S}(t+1) = \begin{cases} \text{rand} \times (UB - LB) + LB, r_1 < z \\ \vec{S}_b(t) + \vec{vb} \times (\vec{W} \times \vec{S}_A(t) - \vec{S}_B(t)), r_2 < p \\ \vec{vc} \times \vec{S}(t), r_2 \geq p \end{cases} \quad (11)$$

其中 r_1 ，和 r_2 是介於0和1之間的兩個隨機值， UB 和 LB 是搜索空間的最大和最小邊界。 z 是概率係數，它決定了當前個體是否尋找另一個食物源以避免陷入局部陷阱，還是利用當前個體尋找更好的解。

2.2.2 黏菌演算法流程

SMA 演算法的運作流程始於一組隨機初始解，接著將迄今為止獲得的最佳解保存為目標點。同時，演算法會更新與目標點相關的其他解。隨著迭代次數的增加，演算法會調整參數 $\vec{vb}$ 和 $\vec{vc}$ 的範圍，以加強對搜索空間的探索。當迭代次數超過預設的最大迭代次數時，SMA 演算法將終止優化過程。有關 SMA 演算法的虛擬程式碼描述可參考圖2。

Algorithm1: Pseudo Code for Slime Mould Algorithm

1. Initialize the parameters pop_size , $Max_iteration$;
 2. Initialize the positions of slime mould $\vec{SM}^i (i = 1, 2, \dots, n)$
 3. **While**($t \leq Max_iteration$)
 4. Calculate the fitness of all slime mould;
 5. Update best fitness F
 6. Calculate the $\vec{W}$ by Eq.(9)
 7. **For** each search portion
 8. Update p , $\vec{vc}$, $\vec{vb}$
 9. update positions by Eq.(11)
 10. **End For**
 11. $t = t + 1$
 12. **End while**
 13. **Return** best Fitness;
-

圖2 SMA 演算法虛擬程式碼

2.3 正弦餘弦演算法

Mirjalili [31]提出了一種基於種群的優化演算法。該演算法透過創建多個隨機候選解，利用正弦餘弦數學特性來求解最佳化問題。SCA 演算法是基於正弦函數和餘弦函數值的變化來實現優化搜索，當正弦函數值或餘弦函數值大於1或者小於-1時，演算法進行全局探勘，當正弦函數值或者餘弦函數值為 $[-1,1]$ 之間，演算法進行局部開發。該演算法還集成了幾個隨機變數和自適應變數，以利於在優化過程中尋找探勘與開發之間的平衡。表2包含了一些近年改良 SCA 演算法的概要。

表2 SCA 演算法的概要

算法名稱	年份	描述
ASCA-DE [32]	2017	ASCA-DE 增強了正弦餘弦演算法，用於結構損傷檢測。
SCA-SVR [33]	2018	提出了一種新的 SCA-SVR 模型，通過使用正弦餘弦演算法來優化 SVR 參數，提高時間序列預測效能。
MSCA [34]	2018	提出改進的正弦餘弦演算法(MSCA)解決電力流優化問題，提高效率和效果。
mSCA [35]	2018	提出改進的正弦餘弦演算法(m-SCA)，解決了低多樣性、陷入局部陷阱和跳過最佳解的問題，有效解決全域優化問題。
MSCA-ELM [36]	2018	提出自動系統，透過 MRI 檢測病理性大腦，結合 MSCA-ELM 方法，優於競爭方案，有望實時辨識具病理特徵的大腦。
ISCA [37]	2019	ISCA 演算法提升 SCA 效能，解決工程及影像處理問題。
DDSCA [38]	2021	提出維度逐維動態正弦餘弦演算法(DDSCA)解決全域優化問題，比較其與其他演算法。

2.3.1 正弦餘弦演算法基本原理

在 SCA 演算法中，首先隨機產生 m 個個體的位置，假設優化問題的每個解對應搜索空間中個體的位置，並用 $X_i = (X_{i1}, X_{i2}, \dots, X_{in})^T$ 表示第 i 個體的位置。其中， n 為個體的維度。當前所有個體經歷的最好位置表示為 $P_i = (P_{b1}, P_{b2}, \dots, P_{bn})^T$ 。在下一次迭代中，第 i 個個體的位置更新分為公式(12)和公式(13)兩個階段：

$$X_i^{t+1} = X_i^t + r_1 \times \sin(r_2) \times |(r_3 P_i^t - X_i^t)| \quad (12)$$

$$X_i^{t+1} = X_i^t + r_1 \times \cos(r_2) \times |(r_3 P_i^t - X_i^t)| \quad (13)$$

其中， X_i^t 表示解在第 i 維和第 t 次迭代中的位置。 P_i 是目標解在第 i 維的位置。 r_2 和 r_3 是 $[0, 1]$ 範圍內的隨機值。 $|\cdot|$ 表示絕對值。結合公式

(12)和(13)。加上隨機參數 r_4 ，其中 r_4 是 $[0, 1]$ 範圍內的隨機值，得到下列公式(14)。

$$X_i^{t+1} = \begin{cases} X_i^t + r_1 \times \sin(r_2) \times |(r_3 P_i^t - X_i^t)| & r_4 < 0.5 \\ X_i^t + r_1 \times \cos(r_2) \times |(r_3 P_i^t - X_i^t)| & r_4 \geq 0.5 \end{cases} \quad (14)$$

從更新公式(14)可以看出，SCA 有四個主要參數 r_1, r_2, r_3, r_4 。其中參數 r_1 表示解的下一個位置是在當前解和目標之間或在它們之外的空間。透過自適應地更改 r_1 參數，它同時考慮了開發和探索。

$$r_1 = a \left(1 - \frac{t}{T} \right) \quad (15)$$

SCA 中兩個階段的 r_1 參數控制由公式(15)提供。其中 t 是當前迭代次數， T 為最大迭代次數， a 為常數值。參數 r_2 為介於 $[0, 2\pi]$ 之間的一個隨機值，其作用是決定下一次迭代時的移動步長。參數 r_3 為介於 $[0, 2]$ 之間的一個隨機權重，參數 r_4 為介於 $[0, 1]$ 之間的一個隨機值，藉由公式(14)提供正弦與餘弦函數之間的轉換。當 $r_4 < 0.5$ 時，依照公式(12)進行位置更新，當 $r_4 \geq 0.5$ 時，依照公式(13)進行位置更新。

由於 Mirjalili 在公式(12)和公式(13)中採用了正弦和餘弦函數，因此他將該演算法稱為 SCA 演算法。正弦和餘弦函數對更新公式(14)的影響如圖3所示，該圖展示了在不同範圍取值時解的搜索區域。從圖3可見，更新公式(14)在搜索空間中定義了兩個位置的搜索範圍。

值得注意的是，儘管圖3中呈現了二維模型，但公式(14)可擴展至更高維度。正弦和餘弦函數的週期性質使得個體能夠在不同位置間進行迭代。這有助於確保開發階段生成的位置介於這兩個位置之間。為了探索更廣泛的解空間，下一個位置應該超出這兩個位置，這可透過調整正弦和餘弦函數的振幅來實現。

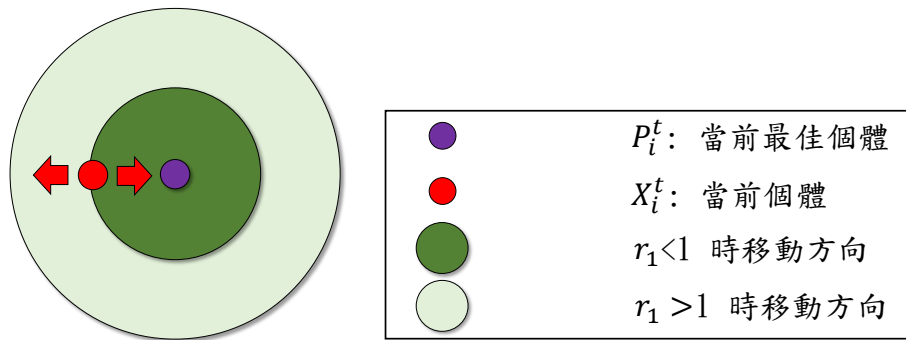


圖3 不同 r_1 的值對解搜索區域的影響

根據圖3所示，在範圍 $[-2,2]$ 內的正弦和餘弦函數對概念模型的影響，從圖4中可以觀察到下一次位置更新是透過調整正弦和餘弦函數的振幅來決定在兩個位置之間或之外的選擇。下一次位置的選擇在兩個位置之間或之外的取得，可透過定義一個隨機參數 r_2 ，其範圍為 $[0,2\pi]$ 。因此，可以確保對搜索空間的探索與開發分別進行。當正弦和餘弦函數的振幅位於 $[-2,-1]$ 和 $[1,2]$ 時，處於探索階段；而當振幅位於 $[-1,1]$ 時，則處於開發階段。

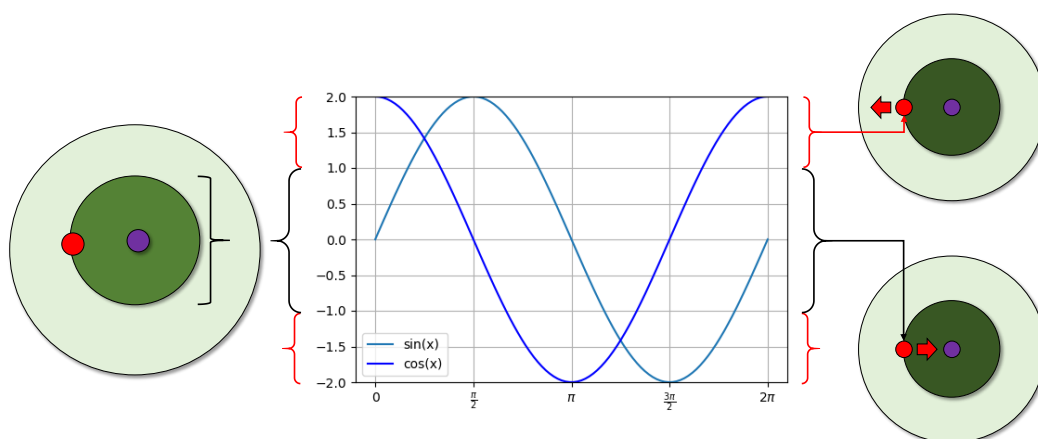


圖4 正弦餘弦演算法的效果

當正弦和餘弦函數的範圍為 $([-2,-1]$ 和 $[1,2])$ 時，SCA 演算法對搜索空間進行探索。而當處於 $[-1,1]$ 的範圍內時，該演算法則對搜索空間進行開發。

2.3.2 正弦餘弦演算法流程

圖5展示了 SCA 演算法的虛擬程式碼描述。SCA 演算法的運作過程始於一組隨機解，然後將最佳解保存至目前為止所獲得的最佳解中，將其指定為目標點，並更新其相關的其他解。同時，調整正弦和餘弦函數的範圍，以突顯隨著迭代次數增加而對搜索空間的開發。當迭代次數超過最大迭代次數時，SCA 演算法將終止優化過程。

Algorithm2: Pseudo Code for Sine Cosine Algorithm

1. *Initialize the search agents*
 2. *Evaluate each search agent using the objective function*
 3. *Select the best solution x_b from the set of search agents*
 4. *Initialize the iteration count $t=0$ and random parameter r_1 ,*
 5. ***While***($t \leq \text{Max_iteration}$)
 6. *For each search agent*
 7. *Update the position of search agent using solution using equations(14)*
 8. *Evaluate the updated search agent*
 9. *Update the best solution x_b*
 10. *Update the parameter r_1 , r_2 and r_3*
 11. ***End For***
 12. $t = t + 1$;
 13. ***End while***
 14. ***Return*** the best Fitness x_b
-

圖5 SCA 演算法虛擬程式碼

第三章 研究方法

黏菌演算法(Slime Mould Algorithm, SMA)和正弦餘弦演算法(Sine Cosine Algorithm, SCA)最初被設計用於處理連續優化問題，然而多維度背包問題(Multi-dimensional Knapsack Problem, MKP)則屬於二元組合優化問題。本研究對 SMA 演算法和 SCA 演算法進行了調整和改良，以解決 MKP 並提出新的混合演算法，即混合黏菌和正弦餘弦演算法(Hybrid Slime Mould and Sine Cosine Algorithm, HSMSCA)。本研究將在後續章節中詳細介紹改良後的 ISMA 演算法、ISCA 演算法以及 HSMSCA 演算法的相關資訊。

3.1 編碼表示方法

MKP 的編碼是二元向量 $x = (x_1, x_2, \dots, x_n)$ 。如果選擇了第 i 項，則 $x_i = 1$ ，否則 $x_i = 0$ 。在 SMA 演算法中，黏菌的位置代表一個解。每個黏菌的位置也是二元的。

3.2 種群初始化方法

在本研究中，我們利用 MKP 的資訊來開發一種生成初始菁英種群的方法。在 MKP 中，物品的性價比率被視為最關鍵的資訊之一，

因為每個物品在產生利潤時都需要消耗多種資源。因此，在尋找最佳的物品子集以最大化總利潤之前，評估物品的效率是必要的。對於經典的背包問題，我們可以輕鬆地通過單位資源消耗所產生的利潤來衡量物品的效率。在 MKP 中，引入了一個相關概念，即物品的性價比率，並提出了多種計算性價比率的方法。其中，基於代理乘數的方法是其中之一，如公式(16)所示。一個好的初始種群可以提高演算法的效率，但如何產生具有多樣性的初始菁英種群取決於要解決的問題

$$u_i = - \frac{p_i}{\sum_{j=1}^m w_{ij} c_i} \quad (16)$$

其中， u_i 代表物品 i 的性價比率； p_i 代表商品 i 的利潤； w_{ij} 代表第 i 個物品佔用第 j 個資源的數量； c_i 代表第 i 個物品的代理乘數，可以視為 MKP 線性鬆弛中第 i 個資源限制的鬆弛解。

圖6展示了生成初始菁英種群方法。為了實現更大的利潤，應首先考慮性價比較高的物品。因此，在不違反任何資源限制的情況下，可以透過隨機選擇性價比較高的物品來產生具有多樣性的初始菁英種群。所有物品 $U = \{i_1, i_2, \dots, i_n \mid u_1 \geq u_2 \geq \dots \geq u_n\}$ 應按照其性價比率的降冪排序。

Algorithm3: Initial elite population generator

```
1. Initialize the search agents
2.  $x_j^k = 0, k = 1, 2, \dots, s; j = 1, 2, \dots, n$ , and the fitness:
3.  $f^k = 0$ 
4. for  $k = 1$  to  $s$  do
5.   Reset the amount of the accumulated resources
6.   consumption:  $r_i = 0, \forall i = 1, 2, \dots, m$ .
7.   for  $j = j_1$  to  $j_n$  do
8.     if  $\text{rand} < 0.5$  and  $\sum_{i=1}^n w_{ji} x_i \leq c_j, j = 0, 1, 2, \dots, m$ 
9.       Then
10.        Set  $x_j^k = 1, f^k \leftarrow f^k + c_j$ , and
11.         $r_i \leftarrow r_i + a_{ij}, \forall i = 1, 2, \dots, m$ .
12.     End
13.   End
14. end
```

圖6 初始精英種群生成的虛擬程式碼

在圖表6中，演算法將根據性價比率降冪排序來選擇物品。在判斷是否接受該物品時，演算法會先生成一個隨機值。如果這個隨機值小於0.5且符合資源限制，則演算法會將代表該物品的決策變數更新為1，同時更新當前維度的剩餘重量。反之，如果隨機值大於等於0.5或不符合資源限制，則演算法會將代表該物品的決策變數設為0，而剩餘容量則保持不變。

3.3 可行解生成方法

在這項研究中，為確保每個個體的位置為二元值，我們採用了tanh轉換函數將連續型數值轉換為0或1的二元變數值。接著，透過修復演算法來確保當前候選解符合所有限制條件，確保其為可行解。

接下來執行兩個階段。首先，按照性價比率的排序逐一捨棄所選擇的物品，直到獲得可行的解。其次，在不違反任何資源限制的情況下，嘗試將每個未選擇的物品(包括第一階段剛剛捨棄的物品)按照性價比率的降序逐一添加到背包中。

3.3.1 轉換函數

轉換函數被視為將任何連續型元啟發式演算法轉換為二元元啟發式演算法的關鍵。元啟發式演算法旨在解決連續型優化問題，其搜索策略無法有效處理離散或二元問題。為解決此問題，連續型演算法引入了 V 型轉換函數，將連續值映射至0和1之間。公式(17)用於標準化結果，將其轉換為代表選擇(1)或不選擇(0)的二元值。

$$F_{bin} = \begin{cases} 1 & \text{if } |\tanh(\vec{S}_i)| \geq \text{rand}() \\ 0 & \text{otherwise} \end{cases} \quad (17)$$

3.3.2 修復演算法

二元解 $\mathbf{z}^k = (z_1^k, z_2^k, \dots, z_n^k)$ ，等式(12)和轉換函數共同產生。存在不可行的情況。為確保最終解決方案的可行性，在修復階段將每個不可行解轉換為可行解，此過程稱為修復演算法。具體做法是按照物品的性價比率降冪排列，逐步從背包中移除物品 $U = \{i_1, i_2, \dots, i_n \mid u_1 \geq u_2 \geq \dots \geq u_n\}$ 直至達到可行解。

Chu 和 Beasley [39]提出的修復演算法已被許多啟發式方法 [10, 14, 40-43]用於解決 MKP。修復演算法充分利用了 MKP 中關於物品價值比率的信息。圖7展示了修復演算法的虛擬程式碼。

Algorithm4: Repair Algorithm(RA)

1. **Input:** *infeasible solution* S_i
 2. **While** $(\sum_{i=1}^n w_{ji}x_i > c_j)$
 3. Remove the items with the lowest u_i
 4. **End while**
 5. **Return** S_i
-

圖7 修復演算法虛擬程式碼

3.3.3 改進演算法

本研究採用了一種改進演算法 [16]，該演算法基於性價比率，旨在降低時間複雜度。文獻指出，在第一階段並非必須檢查每個物品，一旦找到可行解，應立即中斷修復演算法。在處理大規模問題時，改進的時間節省效果將更加顯著。與 Chu 和 Beasley [39]提出的原始改進演算法不同，改進演算法首先檢查是否違反了所有限制條件，然後確定該物品是否應在修復階段中被選中。

根據圖8中修復後的解，改進演算法的虛擬程式碼中所示的改進階段已進行進一步的優化。具體方法包括按照物品 $U = \{i_1, i_2, \dots, i_n \mid u_1 \geq u_2 \geq \dots \geq u_n\}$ 的性價比率進行降冪排序，將性價比率最高的物品優先加入背包中。

Algorithm5: Improvement Algorithm(IA)

1. **Input:** infeasible solution S_i
 2. **for each** items
 3. **If** $(\sum_{i=1}^n w_{ji}x_i \leq c_j)$
 4. **Then**
 5. Insert the items with the height u_i
 6. **End while**
 7. **Return** S_i
-

圖8 改進演算法虛擬程式碼

圖9展示 ISMA 演算法的虛擬程式碼，呈現了 SMA 演算法的改良版本。該演算法首先利用線性鬆弛評估每個物品的性價比率，生成初始的菁英種群，並在每次迭代中，通過修復和改進策略，在個體更新結束時將不可行的解轉換為可用於解決 MKP 的可行解。

Algorithm6: Pseudo Code for Improved Slime Mould Algorithm

1. *Initialize the parameters pop_size , Max_iteration;*
 2. *Initialize the positions of slime mould $\overrightarrow{SM^I} (I = 1, 2, \dots, n)$*
 3. **While**($t \leq \text{Max_iteration}$)
 4. *Calculate the fitness of all slime mould*
 5. *Update best fitness , $\mathbf{BF}$*
 6. *Calculate the $\overrightarrow{W}$ by Eq.(13)*
 7. **For each** search portion
 8. *Update p , $\overrightarrow{vc}$, $\overrightarrow{vb}$*
 9. *update positions by Eq.(15)*
 10. *Apply a transfer function on positions $\vec{S}_i$*
 11. **End For**
 12. *Apply Algorithm 4(RA),and Algorithm 5(IA) on the Binary of $\vec{S}_i$*
 13. $t = t + 1$
 14. **End while**
 15. **Return** best Fitness
-

圖9 ISMA 演算法虛擬程式碼

3.4 改進正弦餘弦演算法

圖10展示了 ISCA 演算法的虛擬程式碼，這是對 SCA 演算法的改進版本。該演算法首先利用線性鬆弛來評估每個物品的性價比，生成初始的優良種群解，並在每次迭代中個體更新結束時，透過修復和改進策略將不可行的解轉換為可用於解決 MKP 的可行解。

Algorithm7: Pseudo Code for Improved Sine Cosine Algorithm

1. *Initialize the search agents*
 2. *Evaluate each search agent using the objective function*
 3. *Select the best solution x_b from the set of search agents*
 4. *Initialize the iteration count $t=0$ and random parameter r_1*
 5. **While**($t \leq \text{Max_iteration}$)
 6. *For each search agent*
 7. *Update the position of search agent using solution using Eq.(14)*
 8. *Evaluate the updated search agent*
 9. *Update the best solution x_b*
 10. *Update the parameter r_1 , r_2 and r_3*
 11. *Apply a transfer function on positions $\vec{S}_i$*
 12. **End For**
 13. *Apply Algorithm 4(RA), and Algorithm 5(IA) on the Binary of $\vec{S}_i$*
 14. $t = t + 1$;
 15. **End while**
 16. **Return** the best Fitness x_b
-

圖10 ISCA 演算法虛擬程式碼

3.5 混合黏菌和正弦餘弦演算法

HSMSCA 演算法的主要步驟如下圖11所示。首先，確定所有必要參數並初始化種群，這代表了問題的候選解。接著，每個解都會經過適應值函數的測試，以評估已知最佳解的品質。隨後，利用強化學

習機制來決定是使用 SMA 演算法還是 SCA 演算法來更新當前個體。這個決定取決於適應值的質量，其使用機率值如公式(18)所示。公式(18)的主要目的在於促使 SMA 演算法和 SCA 演算法競爭性地運作，以增強搜尋過程中勘探和開發之間的平衡。 n 表示已經執行的次數， fit_{iter-1} 表示上一次迭代的適應值。 fit_{iter} 表示當前個體適應值。圖12是 HSMSCA 演算法的虛擬程式碼。

$$Prob = \frac{n-1}{n} \times fit_{iter-1} + \frac{1}{n} \times (fit_{iter} - fit_{iter-1}) / fit_{iter} \quad (18)$$

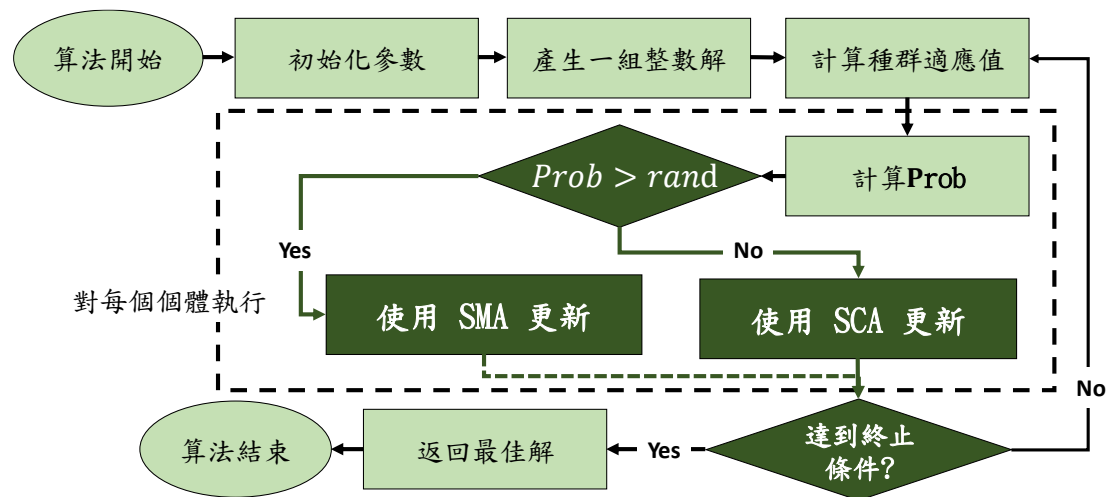


圖11 HSMSCA 演算法流程圖

Algorithm8: PSEUDO Code for Hybrid Slime Mould and Sine Cosine Algorithm

1. Initialize the parameters pop_size , $Max_iteration$
 2. Initialize the positions of slime mould $\vec{SM}^I(I = 1, 2, \dots, n)$
 3. **While**($t \leq Max_iteration$)
 4. update SMA parameter
 5. update SCA parameter
 6. **For** each search portion($\vec{S}_i$)
 7. Choice update method using **Eq.(18)**
 8. **End For**
 9. Apply **Algorithm 4(RA)**, and **Algorithm 5(IA)** on the Binary of $\vec{S}_i$
 10. $t = t + 1$
 11. **End while**
 12. **Return** best Fitness
-

圖12 HSMSCA 演算法虛擬程式碼

3.6 改進黏菌演算法小型 MKP 問題數值示例

假設存在一個簡單問題，其中包含3個資源限制和5個物品。每個物品的資源消耗與各資源限制相關，並根據表4中的利潤分配。該問題的目標在於尋找一個最大化利潤的物品子集，同時滿足背包容量的要求。

表3 小型 MKP 的資源消耗和利潤

物品編號	成本	體積	重量	利潤
1	0	8	3	360
2	0	30	20	83
3	30	98	88	59
4	22	50	50	130
5	80	0	55	431
容量限制	60	65	80	

第一階段，在初始化前先計算該問題的線性鬆弛解 $lp_i = \{1.00, 1.00, 0.00, 0.45, 0.62\}$ ，利用公式(16)計算每個物品的性價比

率。

$$u_1 = -1 \times \frac{360}{(0 + 8 + 3) \times 1.00} = -3.27 \quad (19)$$

$$u_2 = -1 \times \frac{83}{(0 + 30 + 20) \times 1.00} = -1.66 \quad (20)$$

$$u_3 = -1 \times \frac{59}{(30 + 98 + 88) \times 0.0} = -inf \quad (21)$$

$$u_4 = -1 \times \frac{130}{(22 + 50 + 50) \times 0.45} = -2.36 \quad (22)$$

$$u_5 = -1 \times \frac{431}{(80 + 0 + 55) \times 0.62} = -5.14 \quad (23)$$

利用當前鬆弛解，並計算完當前的性價比率後將所有物品 $U = \{i_1, i_2, \dots, i_n \mid u_1 \geq u_2 \geq \dots \geq u_n\}$ 應依其性價比率的降冪排序，表4可以看到最後的排續結果。如果 u_i 出現一個以上的 $-inf$ 值則依照原始序列中的相對排序作為物品選擇的排序。

表4 計算完性價比率後降冪排序

物品編號	成本	體積	重量	利潤	u_i	排序
1	0	8	3	360	-3.27	3
2	0	30	20	83	-1.66	1
3	30	98	88	59	-inf	5
4	22	50	50	130	-2.36	2
5	80	0	55	431	-5.14	4
容量限制	60	65	80			

第二階段，我們使用表4中的排序作為物品選擇的排序，並利用初始菁英種群直到違反限制為止。首先，我們初始化每個維度的當前剩餘容量，然後按排序選擇物品。我們首先將編碼設置為0的陣列，然後對每個物品的編碼元素進行帶有隨機性質的菁英種群生成方法。圖

6展示了該算法的虛擬程式碼。

目前考慮物品 $U = \{i_1, i_2, \dots, i_n \mid u_1 \geq u_2 \geq \dots \geq u_n\}$ ，顯示了針對每個物品的決策機制。見公式(24):

$$S = \left\{ S_i = 1 \mid r < 0.5, \sum_{i=1}^n w_{ji} x_i \leq c_j, j = 0, 1, 2, \dots, m \right\} \quad (24)$$

在公式(24)中，表示第 i 個物品， r 代表隨機值， $\sum_{i=1}^n w_{ji} x_i$ 則表示當前每一維度的累積消耗資源。接著考慮排序中的第一個物品(S_2)，根據公式(25)，當前編碼為 $S = \{0, 1, 0, 0, 0\}$ ，對應的當前利潤為83。考慮排序中的第二個物品(S_4)，參見公式(26):

$$S_2 = 1 \text{ if } 0.6 > 0.5, (0 \leq 60) \wedge (30 \leq 65) \wedge (20 \leq 80) \quad (25)$$

$$S_4 = 1 \text{ if } 0.3 > 0.5, (0 \leq 60) \wedge (30 \leq 65) \wedge (20 \leq 80) \quad (26)$$

由於 r 小於 0.5，因此編碼狀態將繼續考慮下一個排序的物品(S_1)，參見公式(27)。然而，公式(28)違反了限制條件，因此個體的初始解停留在 $S = \{1, 1, 0, 0, 0\}$ ，其利潤為443。

$$S_1 = 1 \text{ if } 0.7 > 0.5, (0 \leq 60) \wedge (38 \leq 65) \wedge (23 \leq 80) \quad (27)$$

$$S_5 = 1 \text{ if } 0.8 > 0.5, (80 \leq 60) \wedge (38 \leq 65) \wedge (78 \leq 80) \quad (28)$$

三階段，我們對 SMA 演算法的位置更新公式(11)進行操作。由於 SMA 演算法的位置更新公式是連續型的，因此更新後的結果不是二元離散解。為了將連續值映射至 $\{0, 1\}$ 範圍內，我們需要使用 $\tanh$ 轉換函數。透過這個轉換，我們可以生成一個隨機值，如果映射值大於

隨機值，則將編碼更新為1。這樣就得到了經過 SMA 位置更新後的候

選解 $S = \{0.34, 0.55, 0.66, 0.98, 0.95\}$

$$S_1 = |\tanh(0.34)| > 0.3 = 1 \quad (29)$$

$$S_2 = |\tanh(0.55)| > 0.4 = 1 \quad (30)$$

$$S_3 = |\tanh(0.66)| > 0.9 = 0 \quad (31)$$

$$S_4 = |\tanh(0.98)| > 0.7 = 1 \quad (32)$$

$$S_5 = |\tanh(0.95)| > 0.8 = 1 \quad (33)$$

經過轉換函數更新後 $S = \{1, 1, 0, 1, 1\}$ ，利潤為1004。同時，檢查當前編碼是否違反限制式，公式(34-36)顯示其計算過程。

$$\text{成本} = 0 \times 1 + 0 \times 1 + 30 \times 0 + 22 \times 1 + 80 \times 1 = 102 (\text{違反限制}) \quad (34)$$

$$\text{體積} = 8 \times 1 + 30 \times 1 + 88 \times 0 + 50 \times 1 + 0 \times 1 = 88 (\text{違反限制}) \quad (35)$$

$$\text{重量} = 3 \times 1 + 20 \times 1 + 88 \times 0 + 50 \times 1 + 55 \times 1 = 128 (\text{違反限制}) \quad (36)$$

第四階段，需要實施修復策略，該策略可分為兩個階段，即修復方法和改進方法。修復方法的目標是將目前不可行的候選解轉換為可行解，具體步驟請參見圖7；而改進方法的目標是將已經修復的可行候選解進一步提升其適應值，具體步驟請參見圖8。

在考慮原始決策變數為1的物品中，首先優先考慮排序最優先的物品 S_5 因此，在目前的情況下，我們將其更新為0，同時更新當前的累積資源消耗和適應值。公式(37-40)展示了這個過程。

$$\text{成本} = 0 \times 1 + 0 \times 1 + 30 \times 0 + 22 \times 1 + 80 \times 0 = 12 \quad (37)$$

$$\text{體積} = 8 \times 1 + 30 \times 1 + 88 \times 0 + 50 \times 1 + 0 \times 0 = 88 (\text{違反限制}) \quad (38)$$

$$\text{重量} = 3 \times 1 + 20 \times 1 + 88 \times 0 + 50 \times 1 + 55 \times 0 = 73 (\text{違反限制}) \quad (39)$$

$$\text{利潤} = 360 \times 1 + 83 \times 1 + 59 \times 0 + 130 \times 1 + 431 \times 0 = 573 \quad (40)$$

因為還是違反限制式因此繼續考慮將 S_1 更新為0

$$\text{成本}=0 \times 0 + 0 \times 1 + 30 \times 0 + 22 \times 1 + 80 \times 0 = 12 \quad (41)$$

$$\text{體積}=8 \times 0 + 30 \times 1 + 88 \times 0 + 50 \times 1 + 0 \times 0 = 80 (\text{違反限制}) \quad (42)$$

$$\text{重量}=3 \times 0 + 20 \times 1 + 88 \times 0 + 50 \times 1 + 55 \times 0 = 70 (\text{違反限制}) \quad (43)$$

$$\text{利潤}=360 \times 0 + 83 \times 1 + 59 \times 0 + 130 \times 1 + 431 \times 0 = 213 \quad (44)$$

因為還是違反限制式因此繼續考慮將 S_4 更新為0

$$\text{成本}=0 \times 0 + 0 \times 1 + 30 \times 0 + 22 \times 0 + 80 \times 0 = 0 \quad (45)$$

$$\text{體積}=8 \times 0 + 30 \times 1 + 88 \times 0 + 50 \times 0 + 0 \times 0 = 30 \quad (46)$$

$$\text{重量}=3 \times 0 + 20 \times 1 + 88 \times 0 + 50 \times 0 + 55 \times 0 = 20 \quad (47)$$

$$\text{利潤}=360 \times 0 + 83 \times 1 + 59 \times 0 + 130 \times 0 + 431 \times 0 = 83 \quad (48)$$

依照表4中的性價比率排序，進行改進方法以優先考慮原先決策變數為0中排序最優先的物品(S_4)在將該物品退回至上一個狀態後，依序遍歷所有物品直至序列的最後一個物品。透過上述步驟，完成了從評估性價比率到更新位置後修復的流程。

$$\text{成本}=0 \times 0 + 0 \times 1 + 30 \times 0 + 22 \times 1 + 80 \times 0 = 22 \quad (49)$$

$$\text{體積}=8 \times 0 + 30 \times 1 + 88 \times 0 + 50 \times 1 + 0 \times 0 = 80 (\text{違反限制}) \quad (50)$$

$$\text{重量}=3 \times 0 + 20 \times 1 + 88 \times 0 + 50 \times 1 + 55 \times 0 = 70 (\text{違反限制}) \quad (51)$$

$$\text{利潤}=360 \times 0 + 83 \times 1 + 59 \times 0 + 130 \times 1 + 431 \times 0 = 213 \quad (52)$$

第四章 實驗結果

在本節中，進行了大量實驗，以評估提出的演算法的效能。還將所提演算法的效能與其他競爭演算法進行了對比測試。

4.1 演算法開發的環境

在一部搭載 Windows 11作業系統、32 GB 記憶體和 Intel i7處理器的電腦上進行了實驗，所提出的演算法求解全部題庫20次。該演算法是在 Visual Studio Code 編譯環境中，使用 Python 3.11語言實現的。實驗的終止條件為個體數量20，進行了100,000次評估 [19]。

4.2 題庫來源

本文使用四組經典的 MKP 題庫來測試演算法的效能。這些題庫在文獻中有所描述，可在 OR-Library 中獲取 (<http://people.brunel.ac.uk/mastjjb/jeb/orlib/mknapinfo.html>)。

題庫1包含16個小規模個題目，其中有 $m=2$ 至30和 $n=20$ 至105，Sento [44]、HP [45]、PB [45]、Weing [46]。

題庫2包含30個中等規模題目，其中 $m=5$ 和 $n=30$ 至90的中等規模題目，這些題目被標記為 Weish [47]。

題庫3包含30個大規模題目。這些題目分為兩個子集。子集3-1包括15個具有 $m=15$ 和 $n \in \{100, 250, 500\}$ 的15個題目，分別標記為 cb1、cb2和 cb3。子集3-2還包括15個具有 $m=15$ 和 $n \in \{100, 250, 500\}$ 的15個題目，分別標記為 cb4、cb5和 cb6 [39]。

題庫4包含9個題目，其中 $m = \{15, 25, 50\}$ 和 $n \in \{100, 200, 500, 1000, 1500\}$ 的題目，這些題目由 Glover 和 Kochenberger 創建，並標記為 GK [48]。

表5詳細描述 Sento、HP、PB、Weing、WEISH、CB 和 GK 題庫。

BKS 一欄顯示的是每個題目的已知最佳值 [19]。

表5 題庫規模

	題號	BKS	數量	維度	題號	BKS	數量	維度	題號	BKS	數量	維度
題庫 1	Sento 1	7772	60	30	PB4	95168	29	2	Weing3	95677	28	2
	Sento 2	8722	60	30	PB5	2139	20	10	Weing4	119337	28	2
	HP1	3418	28	4	PB6	776	40	30	Weing5	98796	28	2
	HP2	3186	35	4	PB7	1035	37	30	Weing6	130623	28	2
	PB1	3090	27	4	Weing1	141278	28	30	Weing7	1095445	105	2
	PB2	3186	34	4	Weing2	130883	28	2	Weing8	624319	105	2
	題號	BKS	數量	維度	題號	BKS	數量	維度	題號	BKS	數量	維度
題庫 2	weish01	4554	30	5	weish11	5643	50	5	weish21	9074	70	5
	weish02	4536	30	5	weish12	6339	50	5	weish22	3947	70	5
	weish03	4115	30	5	weish13	6159	50	5	weish23	8344	80	5
	weish04	4561	30	5	weish14	6954	60	5	Weish24	10,220	80	5
	weish05	4514	30	5	weish15	7436	60	5	Weish25	9939	80	5
	weish06	5557	40	5	weish16	7289	60	5	Weish26	9584	90	5
	weish07	5567	40	5	weish17	8633	60	5	weish27	9819	90	5
	weish08	5605	40	5	weish18	9580	70	5	weish28	9492	90	5
	weish09	5246	40	5	weish19	7698	70	5	weish29	9410	90	5
	weish10	6339	50	5	weish20	9450	70	5	weish30	11,191	90	5
	題號	BKS	數量	維度	題號	BKS	數量	維度	題號	BKS	數量	維度
題庫 3	cb1-1	24381	100	5	cb3-1	120148	500	5	cb5-1	59187	250	10
	cb1-2	24274	100	5	cb3-2	117879	500	5	cb5-2	58781	250	10
	cb1-3	23551	100	5	cb3-3	121131	500	5	cb5-3	58097	250	10
	cb1-4	23534	100	5	cb3-4	120804	500	5	cb5-4	61000	250	10
	cb1-5	23991	100	5	cb3-5	122319	500	5	cb5-5	58092	250	10
	cb 2-1	59312	250	5	cb4-1	23064	100	10	bb6-1	117821	500	10
	cb 2-2	61472	250	5	cb4-2	22801	100	10	cb6-2	119249	500	10
	cb 2-3	62130	250	5	cb4-3	22131	100	10	cb6-3	119215	500	10
	cb 2-4	59463	250	5	cb4-4	22772	100	10	cb6-4	118829	500	10
	cb 2-5	58951	250	5	cb4-5	22751	100	10	cb6-5	116530	500	10
	題號	BKS	數量	維度	題號	BKS	數量	維度	題號	BKS	數量	維度
題庫 4	GK-1	3766	100	15	GK-4	5767	150	50	GK-7	19221	500	25
	GK-2	3958	100	25	GK-5	7560	200	25	GK-8	18806	500	50
	GK-3	5656	150	25	GK-6	7677	200	50	GK-9	58089	1500	25

4.3 比較對象與評估準則

本研究選擇了近年文獻中多種 MKP 演算法作為競爭演算法，包括混合學習蛾搜索演算法(Hybrid Learning Moth Search, HLMS) [19]、正弦餘弦演算法(Sine Cosine Algorithm, SCA) [49]、黏菌演算法(Slime Mould Algorithm, SMA) [18]。鯨魚優化演算法[15]、改良多宇宙優化演算法(Modified Multi-Verse Optimization, MMVO)[17]。

競爭演算法的結果是根據相關文獻整理而來。若某個 MKP 題庫的演算法結果無法使用，則將該問題的結果排除在外。考慮到不同的比較演算法採用不同的程式設計語言撰寫，或基於不同的終止條件和演算法參數在不同的計算平臺上執行，我們將著重於比較平均百分比偏差。

本研究使用三種統計指標來評估所提出之演算法與競爭對手演算法的效能，包括最佳值、平均值以及百分比偏差(PDev)。

百分比偏差(PDev)是一項統計值，旨在評估最佳已知解(Best Known Solution, BKS)與所獲解平均值之間的誤差百分比。當平均 PDev 值最低時，該演算法被視為最佳演算法。PDev 的計算公式(53)：

$$PDev = \frac{BKS - Mean}{BKS} * 100 \quad (53)$$

4.4 參數實驗

在本節中，將對參數的影響進行了分析。首先，基於每一個題庫中隨機選擇兩題進行了驗證，實驗結果匯總結果如表 7 所示。對每一題的 PDev 求平均值作為評估該參數的表現。其次，設定參數組合 ISMA 演算法的 $r=\{0.01,0.07,0.15\}$ 、ISCA 演算法的 $r_1=\{0.15,2.0,2.5\}$ 、HSMSCA 演算法的參數組合由 $r=\{0.01, 0.07, 0.15\}$ 與 $r_1=\{0.15,2.0,2.5\}$ 構成。不同參數組合的演算法在隨機選擇的題庫中評估結果，附錄 A 列出所有參數的最佳、平均和 PDev。可以得出的結論分別為 ISMA 演算法的 $r=0.08$ 、ISCA 演算法的 $r_1=1.5$ 、HSMSCA 演算法的 $r = 0.08, r_1 = 2.5$ 表現為最佳。評估的方法是將所有隨機選擇題庫的 PDev 平均值進行比較。實驗的終止條件為個體數量 20，進行了 100,000 次評估 [19]。

表6 參數實驗結果

演算法	ISMA				ISCA				
	$r = 0.01$	$r = 0.08$	$r = 0.15$	$r_1 = 1.5$	$r_1 = 2.0$	$r_1 = 2.5$	-	-	-
平均 PDev	0.072	0.065	0.073	0.066	0.094	0.114	-	-	-
演算法	HSMSCA								
參數	$r = 0.01, r_1 = 1.5$	$r = 0.01, r_1 = 2.0$	$r = 0.01, r_1 = 2.5$	$r = 0.08, r_1 = 1.5$	$r = 0.08, r_1 = 2.0$	$r = 0.08, r_1 = 2.5$	$r = 0.15, r_1 = 1.5$	$r = 0.15, r_1 = 2.0$	$r = 0.15, r_1 = 2.5$
平均 PDev	0.081	0.068	0.067	0.066	0.064	0.063	0.067	0.065	0.071

表中數值為平均 PDev 結果

4.5 比較結果

在本節中，採用前一節得到的最佳參數值(ISMA 的 $r = 0.01$ 、ISCA 的 $r_1 = 2.0$ 和 HSMSCA 的 $r = 0.15, r_1 = 2.0$)與一些相關演算法進行比較。如上所述，所有的演算法求解全部題庫 20 次，附錄 B 詳細記錄每種演算法的效能。

結果匯總結果如表 7 所示。ISMA 的 $r = 0.01$ 、ISCA 的 $r_1 = 2.0$ 和 HSMSCA 的 $r = 0.15, r_1 = 2.0$ 與其他演算法進行相比，優勢明顯。另外，對於所有其他的題庫，所提出的 HSMSCA 演算法的表現也比其他算法好。

我們對題庫 1 的 18 個小型題庫中提出的演算法進行了驗證與比較，具體結果見附錄 B。比較的指標包括最大值、平均值和標準差百分比。其中，表 7 列出了 Pdev 的平均值，並將最佳結果以粗體突出表示。

在題庫 2 的實驗中，所提演算法與其他五種演算法(HLMS、IBSMA_U1、SCA、BIWOA、BMMVO)進行了比較。效能評估的具體數據則在表 7 中顯示，其中 HSMSCA 在 30 個題目中均取得了最佳解。

對於題庫 3，表 7 總結了相關的實驗結果。在 3-1 中與其他兩種演算法比較(HLMS、BIWOA)，在 3-2 中與 HLMS 比較，從數據可以看出，

在題庫3-1與3-2中，HSMSCA 的表現輕微領先於 ISMA 和 ISCA，並顯著優於其他演算法。

至於題庫4，同樣對所提出的演算法進行了驗證與比較，結果也是依照 Max/Mean/Pdev/Rank 來排列，表7再次列出了 Pdev 與 Rank 的平均值，並將最佳結果以粗體標出。

表7 比較與評估結果

題庫	HSMSCA	ISMA	ISCA	HLMS	BIWOA	BMMVO	BSCA	IBSMA_U1
題庫 1	0.020/1.167	0.023/ 1.167	0.200/2.444	0.569/3.556	-/-	-/-	-/-	-/-
題庫 2	0.000/1.000	0.000/1.000	0.050/2.433	0.154/4.367	0.472/4.033	0.861/5.133	0.314/3.933	0.105/2.467
題庫 3-1	0.091/1.800	0.097/2.533	0.096/1.867	0.925/4.267	1.234/4.533	-/-	-/-	-/-
題庫 3-2	0.219/1.800	0.225/2.333	0.233/1.933	1.349/3.933	-/-	-/-	-/-	-/-
題庫 4	0.180/1.778	0.182/2.222	0.188/2.000	0.922/5.444	0.857/5.333	2.438/8.000	1.997/7.000	0.624/4.111

平均 PDev/Rank

第五章 結論與未來研究方向

5.1 結論

本研究採用 SMA 與 SCA 的二元版本，並運用線性鬆弛方法處理 MKP 問題。此外，透過結合這兩種演算法形成的新型混合演算法 HMSCA，並引入強化學習機制以最佳化編碼更新策略。本研究利用線性鬆弛解來計算物品的性價比率，不僅評估物品選擇的偏好，同時有效引導搜索過程，從而顯著提高解的尋找效率。提出了一個創新的演算法框架，該框架利用強化學習機制來決定是採用 SMA 或 SCA 演算法更新編碼，以增強演算法的適應性和效能。

HMSCA 演算法的有效性已經透過常見的四種規模的題庫在解決 MKP 方面得到驗證。研究結果指出，相較於 ISMA、ISCA 演算法以及其他元啟發式演算法，HMSCA 展現出較佳的效能。

5.2 未來研究方向

未來對於 HMSCA 演算法的研究可分為兩個主要方向：一是探索其他最佳化問題，另一是進一步改進演算法。在最佳化問題方面，HMSCA 演算法有潛力解決更多最佳化問題，例如多目標背包問題

(Multi-Objective Knapsack Problems, MOKP)以及機器學習的特徵篩選。在演算法改進方面，可考慮引入正交學習、深度強化學習以及自適應學習策略等方法，以進一步增強演算法的搜索能力。

參考文獻

- [1] Toyoda, Y., *A simplified algorithm for obtaining approximate solutions to zero-one programming problems*. Management Science, 1975. **21**(12): p. 1417-1427.
- [2] Vasquez, M. and Y. Vimont, *Improved results on the 0–1 multidimensional knapsack problem*. European Journal of Operational Research, 2005. **165**(1): p. 70-81.
- [3] Feng, Y., et al., *Solving 0–1 knapsack problems by chaotic monarch butterfly optimization algorithm with Gaussian mutation*. Memetic Computing, 2018. **10**: p. 135-150.
- [4] Li, Y., Y. Tao, and F. Wang, *A compromised large-scale neighborhood search heuristic for capacitated air cargo loading planning*. European Journal of Operational Research, 2009. **199**(2): p. 553-560.
- [5] Pisinger, D., *Heuristics for the container loading problem*. European Journal of operational research, 2002. **141**(2): p. 382-392.
- [6] Engwall, M. and A. Jerbrant, *The resource allocation syndrome: the prime challenge of multi-project management?* International Journal of project management, 2003. **21**(6): p. 403-409.
- [7] Johari, R. and J.N. Tsitsiklis, *Efficiency loss in a network resource allocation game*. Mathematics of Operations Research, 2004. **29**(3): p. 407-435.
- [8] Balev, S., et al., *A dynamic programming based reduction procedure for the multidimensional 0–1 knapsack problem*. European Journal of operational research, 2008. **186**(1): p. 63-76.
- [9] Boyer, V., D. El Baz, and M. Elkihel, *Solution of multidimensional knapsack problems via cooperation of dynamic programming and branch and bound*. European Journal of Industrial Engineering, 2010. **4**(4): p. 434-449.
- [10] Puchinger, J., G.R. Raidl, and U. Pferschy, *The multidimensional knapsack problem: Structure and algorithms*. INFORMS Journal on Computing, 2010. **22**(2): p. 250-265.
- [11] Vimont, Y., S. Boussier, and M. Vasquez, *Reduced costs propagation in an efficient implicit enumeration for the 01 multidimensional knapsack problem*. Journal of Combinatorial Optimization, 2008. **15**(2): p. 165-178.
- [12] Du, D., K.-I. Ko, and X. Hu, *Design and analysis of approximation*

algorithms. Vol. 62. 2012: Springer.

[13]Motwani, R. and P. Raghavan, *Randomized algorithms*. 1995: Cambridge university press.

[14]Haddar, B., et al., *A hybrid quantum particle swarm optimization for the multidimensional knapsack problem*. Engineering Applications of Artificial Intelligence, 2016. **55**: p. 1-13.

[15]Abdel-Basset, M., D. El-Shahat, and A.K. Sangaiah, *A modified nature inspired meta-heuristic whale optimization algorithm for solving 0–1 knapsack problem*. International Journal of Machine Learning and Cybernetics, 2019. **10**: p. 495-514.

[16]Luo, K. and Q. Zhao, *A binary grey wolf optimizer for the multidimensional knapsack problem*. Applied Soft Computing, 2019. **83**: p. 105645.

[17]Abdel-Basset, M., et al., *A binary multi-verse optimizer for 0-1 multidimensional knapsack problems with application in interactive multimedia systems*. Computers & Industrial Engineering, 2019. **132**: p. 187-206.

[18]Abdel-Basset, M., et al., *BSMA: A novel metaheuristic algorithm for multi-dimensional knapsack problems: Method and comprehensive analysis*. Computers & Industrial Engineering, 2021. **159**: p. 107469.

[19]Feng, Y., et al., *Hybrid Learning Moth Search Algorithm for Solving Multidimensional Knapsack Problems*. Mathematics, 2023. **11**(8): p. 1811.

[20]Li, S., et al., *Slime mould algorithm: A new method for stochastic optimization*. Future Generation Computer Systems, 2020. **111**: p. 300-323.

[21]Kumar, C., et al., *A new stochastic slime mould optimization algorithm for the estimation of solar photovoltaic cell parameters*. Optik, 2020. **223**: p. 165277.

[22]Zubaidi, S.L., et al., *Hybridised artificial neural network model with slime mould algorithm: a novel methodology for prediction of urban stochastic water demand*. Water, 2020. **12**(10): p. 2692.

[23]Chen, Z. and W. Liu, *An efficient parameter adaptive support vector regression using K-means clustering and chaotic slime mould algorithm*. Ieee Access, 2020. **8**: p. 156851-156862.

[24]Houssein, E.H., et al., *Hybrid slime mould algorithm with adaptive guided differential evolution algorithm for combinatorial and global optimization problems*. Expert Systems with Applications, 2021. **174**: p. 114689.

[25]Ahmadianfar, I., et al., *Multi-strategy Slime Mould Algorithm for*

hydropower multi-reservoir systems optimization. Knowledge-Based Systems, 2022. **250**: p. 109048.

[26]Houssein, E.H., et al., *An efficient orthogonal opposition-based learning slime mould algorithm for maximum power point tracking*. Neural Computing and Applications, 2022: p. 1-25.

[27]Alfadhli, J., et al., *FP-SMA: an adaptive, fluctuant population strategy for slime mould algorithm*. Neural Computing and Applications, 2022. **34**(13): p. 11163-11175.

[28]Rizk-Allah, R.M., A.E. Hassanien, and D. Song, *Chaos-opposition-enhanced slime mould algorithm for minimizing the cost of energy for the wind turbines on high-altitude sites*. ISA transactions, 2022. **121**: p. 191-205.

[29]Örnek, B.N., et al., *A novel version of slime mould algorithm for global optimization and real world engineering problems: Enhanced slime mould algorithm*. Mathematics and Computers in Simulation, 2022. **198**: p. 253-288.

[30]Ewees, A.A., et al., *Improved Slime Mould Algorithm based on Firefly Algorithm for feature selection: A case study on QSAR model*. Engineering with Computers, 2021: p. 1-15.

[31]Mirjalili, S., *SCA: a sine cosine algorithm for solving optimization problems*. Knowledge-based systems, 2016. **96**: p. 120-133.

[32]Bureerat, S. and N. Pholdee. *Adaptive sine cosine algorithm integrated with differential evolution for structural damage detection*. in *Computational Science and Its Applications–ICCSA 2017: 17th International Conference, Trieste, Italy, July 3-6, 2017, Proceedings, Part I 17*. 2017. Springer.

[33]Li, S., H. Fang, and X. Liu, *Parameter optimization of support vector regression based on sine cosine algorithm*. Expert systems with Applications, 2018. **91**: p. 63-77.

[34]Attia, A.-F., R.A. El Sehiemy, and H.M. Hasanien, *Optimal power flow solution in power systems using a novel Sine-Cosine algorithm*. International Journal of Electrical Power & Energy Systems, 2018. **99**: p. 331-343.

[35]Gupta, S. and K. Deep, *A hybrid self-adaptive sine cosine algorithm with opposition based learning*. Expert Systems with Applications, 2019. **119**: p. 210-230.

[36]Nayak, D.R., et al., *Combining extreme learning machine with modified sine cosine algorithm for detection of pathological brain*.

- Computers & Electrical Engineering, 2018. **68**: p. 366-380.
- [37]Gupta, S. and K. Deep, *Improved sine cosine algorithm with crossover scheme for global optimization*. Knowledge-Based Systems, 2019. **165**: p. 374-406.
- [38]Li, Y., Y. Zhao, and J. Liu, *Dimension by dimension dynamic sine cosine algorithm for global optimization problems*. Applied Soft Computing, 2021. **98**: p. 106933.
- [39]Chu, P.C. and J.E. Beasley, *A genetic algorithm for the multidimensional knapsack problem*. Journal of heuristics, 1998. **4**: p. 63-86.
- [40]Zhang, B., et al., *An effective hybrid harmony search-based algorithm for solving multidimensional knapsack problems*. Applied Soft Computing, 2015. **29**: p. 288-297.
- [41]Azad, M.A.K., A.M.A. Rocha, and E.M. Fernandes, *Improved binary artificial fish swarm algorithm for the 0–1 multidimensional knapsack problems*. Swarm and evolutionary computation, 2014. **14**: p. 66-75.
- [42]Zhang, S., et al., *Grey wolf optimizer for unmanned combat aerial vehicle path planning*. Advances in Engineering Software, 2016. **99**: p. 121-136.
- [43]Meng, T. and Q.-K. Pan, *An improved fruit fly optimization algorithm for solving the multidimensional knapsack problem*. Applied Soft Computing, 2017. **50**: p. 79-93.
- [44]Senju, S. and Y. Toyoda, *An approach to linear programming with 0-1 variables*. Management Science, 1968: p. B196-B207.
- [45]Freville, A., *Hard 0-1 multik-napsack test problems for size reduction methods*. Investigation Operativa, 1990.
- [46]Weingartner, H.M. and D.N. Ness, *Methods for the solution of the multidimensional 0/1 knapsack problem*. Operations Research, 1967. **15**(1): p. 83-103.
- [47]Shih, W., *A branch and bound method for the multiconstraint zero-one knapsack problem*. Journal of the Operational Research Society, 1979. **30**: p. 369-378.
- [48]Glover, F. and G.A. Kochenberger, *Critical event tabu search for multidimensional knapsack problems*. Meta-heuristics: Theory and applications, 1996: p. 407-427.
- [49]Pinto, H., et al. *A binary sine-cosine algorithm applied to the knapsack problem*. in *Artificial Intelligence Methods in Intelligent Algorithms: Proceedings of 8th Computer Science On-line Conference 2019, Vol. 2* 8.

2019. Springer.

附錄

附錄 A 參數實驗

	SMA 參數(z) = 0.01	SMA 參數(z) = 0.08	SMA 參數(z) = 0.15	SCA 參數(r1) = 1.5	SCA 參數(r1) = 2.0	SCA 參數(r1) = 2.5	參數(z) = 0.01, (r1) = 1.5	參數(z) = 0.01, (r1) = 2.0
GK-4	5760/5751.200/0.270	5760/5753.450/0.230	5758/5752.150/0.260	5758/5749.400/0.310	5758/5748.850/0.310	5758/5749.950/0.300	5754/5746.500/0.360	5759/5749.100/0.310
GK-9	58073/58069.650/0.030	58071/58064.250/0.040	58071/58061.500/0.050	58077/58070.450/0.030	58074/58070.950/0.030	58075/58070.150/0.030	58074/58065.850/0.040	58076/58071.650/0.030
cb3-3	121121/121053.200/0.060	121121/121057.250/0.060	121115/121034.250/0.080	121121/121088.550/0.040	121121/121093.750/0.030	121121/121091.600/0.030	121109/121038.500/0.080	121121/121078.100/0.040
cb6-2	119099/119057.150/0.160	119087/119029.600/0.180	119082/119015.450/0.200	119114/119083.950/0.140	119179/119088.950/0.130	119108/119082.950/0.140	119139/119063.150/0.160	119086/119082.200/0.140
Weish16	7289/7287.900/0.020	7289/7289.000/0.000	7289/7289.000/0.000	7289/7288.000/0.010	7289/7287.700/0.020	7289/7287.900/0.020	7289/7287.400/0.020	7289/7287.800/0.020
Weish26	9584/9581.400/0.030	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000
PB4	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/94955.050/0.220	95168/94793.400/0.390	95168/95168.000/0.000	95168/95168.000/0.000
Weing5	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000
	參數(z) = 0.01,(r1) = 2.5	參數(z) = 0.08,(r1) = 1.5	參數(z) = 0.08,(r1) = 2.0	參數(z) = 0.08,(r1) = 2.5	參數(z) = 0.15,(r1) = 1.5	參數(z) = 0.15,(r1) = 2.0	參數(z) = 0.15,(r1) = 2.5	
GK-4	5756/5749.650/0.300	5758/5750.450/0.290	5757/5751.100/0.280	5759/5751.650/0.270	5758/5751.450/0.270	5757/5751.350/0.270	5758/5750.650/0.280	
GK-9	58077/58070.600/0.030	58074/58066.850/0.040	58077/58068.400/0.040	58075/58068.500/0.040	58070/58066.000/0.040	58073/58066.550/0.040	58073/58066.250/0.040	
cb3-3	121109/121071.600/0.050	121106/121056.500/0.060	121093/121056.750/0.060	121121/121057.450/0.060	121109/121044.450/0.070	121121/121053.650/0.060	121109/121054.500/0.060	
cb6-2	119082/119080.450/0.140	119082/119076.350/0.140	119082/119078.000/0.140	119105/119079.950/0.140	119087/119064.600/0.150	119087/119071.200/0.150	119087/119036.200/0.180	
Weish16	7289/7288.100/0.010	7289/7289.000/0.000	7289/7289.000/0.000	7289/7289.000/0.000	7289/7289.000/0.000	7289/7289.000/0.000	7289/7289.000/0.000	
Weish26	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	9584/9584.000/0.000	
PB4	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	95168/95168.000/0.000	
Weing5	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	98796/98796.000/0.000	

Max/ Mean/ Pdev

附錄 B 題庫結果

題庫	ISMA	ISCA	HSMSCA	HLMS	BIWOA	BMMVO	BSCA	IBSMA_U1
Sento 1	7772/7772/0.000/1	7772/7762/0.134/4	7772/7772/0.000/1	-/7762.000/0.129/3	-/-/-	-/-/-	-/-/-	-/-/-
Sento 2	8722/8722/0.000/1	8722/8722/0.000/1	8722/8722/0.000/1	-/8719.000/0.034/4	-/-/-	-/-/-	-/-/-	-/-/-
HP1	3418/3416/0.061/2	3418/3414/0.123/3	3418/3417/0.020/1	-/3399.000/0.556/4	-/-/-	-/-/-	-/-/-	-/-/-
HP2	3186/3183/0.085/2	3186/3176/0.317/3	3186/3185/0.028/1	-/3158.000/0.879/4	-/-/-	-/-/-	-/-/-	-/-/-
PB1	3090/3090/0.000/1	3090/3075/0.492/3	3090/3090/0.000/1	-/3074.000/0.518/4	-/-/-	-/-/-	-/-/-	-/-/-
PB2	3186/3184/0.056/2	3186/3171/0.476/4	3186/3185/0.028/1	-/3178.000/0.251/3	-/-/-	-/-/-	-/-/-	-/-/-
PB4	95168/95168/0.000/1	95168/94955/0.224/3	95168/95168/0.000/1	-/93063.000/2.212/4	-/-/-	-/-/-	-/-/-	-/-/-
PB5	2139/2139/0.000/1	2139/2139/0.000/1	2139/2139/0.000/1	-/2125.000/0.655/4	-/-/-	-/-/-	-/-/-	-/-/-
PB6	776/774/0.213/1	765/765/1.418/4	776/774/0.213/1	-/768.000/1.031/3	-/-/-	-/-/-	-/-/-	-/-/-
PB7	1035/1035/0.000/1	1035/1035/0.000/1	1035/1035/0.000/1	-/1030.000/0.483/4	-/-/-	-/-/-	-/-/-	-/-/-

Weing1	141278/141278/0.000/1	141278/141278/0.000/1	141278/141278/0.000/1	-/141260.000/0.013/4	-/-	-/-	-/-	-/-
Weing2	130883/130883/0.000/1	130883/130763/0.092/4	130883/130883/0.000/1	-/130877.000/0.005/3	-/-	-/-	-/-	-/-
Weing3	95677/95677/0.000/1	95677/95588/0.093/3	95677/95662/0.016/2	-/94801.000/0.916/4	-/-	-/-	-/-	-/-
Weing4	119337/119337/0.000/1	119337/119337/0.000/1	119337/119337/0.000/1	-/118956.000/0.319/4	-/-	-/-	-/-	-/-
Weing5	98796/98796/0.000/1	98796/98796/0.000/1	98796/98796/0.000/1	-/98796.000/0.000/1	-/-	-/-	-/-	-/-
Weing6	130623/130623/0.000/1	130623/130331/0.224/4	130623/130623/0.000/1	-/130610.000/0.010/3	-/-	-/-	-/-	-/-
Weing7	1095445/1095385/0.005/1	1095382/1095382/0.006/2	1094757/1094757/0.063/3	-/1073783.000/1.977/4	-/-	-/-	-/-	-/-
Weing8	624319/624319/0.000/1	624319/624319/0.000/1	624319/624319/0.000/1	-/622775.000/0.247/4	-/-	-/-	-/-	-/-
Weish01	4554/-/0.000/1	4554/-/0.000/1	4554/-/0.000/1	4554/-/0.180/7	4554/-/0.000/1	4554/-/0.000/1	4554/-/0.090/6	4554/-/0.000/1
Weish02	4536/-/0.000/1	4536/-/0.298/7	4536/-/0.000/1	4536/-/0.020/6	4536/-/0.000/1	4536/-/0.000/1	4536/-/0.000/1	4536/-/0.000/1
Weish03	4115/-/0.000/1	4115/-/0.077/4	4115/-/0.000/1	4115/-/0.190/5	4106/-/3.970/7	4106/-/3.050/6	4106/-/0.000/1	4115/-/0.000/1
Weish04	4561/-/0.000/1	4561/-/0.000/1	4561/-/0.000/1	4561/-/0.090/7	4561/-/0.000/1	4561/-/0.000/1	4561/-/0.000/1	4561/-/0.000/1
Weish05	4514/-/0.000/1	4514/-/0.060/7	4514/-/0.000/1	4514/-/0.000/1	4514/-/0.000/1	4514/-/0.000/1	4514/-/0.000/1	4514/-/0.000/1
Weish06	5557/-/0.000/1	5557/-/0.000/1	5557/-/0.000/1	5557/-/0.090/4	5557/-/0.120/5	5557/-/0.140/6	5557/-/0.140/6	5557/-/0.000/1
Weish07	5567/-/0.000/1	5567/-/0.000/1	5567/-/0.000/1	5567/-/0.030/7	5567/-/0.000/1	5567/-/0.000/1	5567/-/0.000/1	5567/-/0.000/1
Weish08	5605/-/0.000/1	5605/-/0.000/1	5605/-/0.000/1	5605/-/0.010/5	5605/-/0.000/1	5605/-/0.030/7	5605/-/0.010/5	5605/-/0.000/1
Weish09	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1	5246/-/0.000/1
Weish10	6339/-/0.000/1	6339/-/0.000/1	6339/-/0.000/1	6339/-/0.040/4	6323/-/0.310/5	6303/-/0.560/6	6303/-/0.560/6	6323/-/0.505/6
Weish11	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1	5643/-/0.000/1
Weish12	6339/-/0.000/1	6339/-/0.000/1	6339/-/0.000/1	6339/-/0.070/4	6302/-/0.580/5	6301/-/0.590/6	6302/-/0.590/6	6302/-/0.584/6
Weish13	6159/-/0.000/1	6159/-/0.000/1	6159/-/0.000/1	6159/-/0.230/7	6159/-/0.000/1	6159/-/0.000/1	6159/-/0.000/1	6159/-/0.000/1
Weish14	6954/-/0.000/1	6954/-/0.000/1	6954/-/0.000/1	6954/-/0.280/4	6923/-/0.710/7	6923/-/0.570/5	6923/-/0.660/6	6923/-/0.479/5
Weish15	7486/-/0.000/1	7486/-/0.075/4	7486/-/0.000/1	7486/-/0.080/5	7486/-/0.080/5	7486/-/0.110/7	7486/-/0.000/1	7486/-/0.000/1
Weish16	7289/-/0.000/1	7289/-/0.014/4	7289/-/0.000/1	7289/-/0.140/7	7289/-/0.010/3	7289/-/0.100/6	7289/-/0.090/5	7289/-/0.003/3
Weish17	8633/-/0.000/1	8633/-/0.073/4	8633/-/0.000/1	8633/-/0.000/1	8633/-/0.390/5	8624/-/0.960/7	8633/-/0.420/6	8633/-/0.014/4
Weish18	9580/-/0.000/1	9580/-/0.000/1	9580/-/0.000/1	9580/-/0.110/4	9560/-/0.650/6	9456/-/1.920/7	9573/-/0.620/5	9580/-/0.044/4
Weish19	7698/-/0.000/1	7698/-/0.093/5	7698/-/0.000/1	7698/-/0.010/4	7698/-/0.380/7	7698/-/0.350/6	7698/-/0.000/1	7698/-/0.000/1
Weish20	9450/-/0.000/1	9450/-/0.057/5	9450/-/0.000/1	9450/-/0.030/4	9450/-/0.230/6	9445/-/0.570/7	9450/-/0.020/3	9450/-/0.000/1
Weish 21	9074/-/0.000/1	9074/-/0.000/1	9074/-/0.000/1	9074/-/0.050/5	9074/-/0.190/6	9074/-/0.640/7	9074/-/0.030/4	9074/-/0.000/1
Weish22	8947/-/0.000/1	8947/-/0.386/3	8947/-/0.000/1	8929/-/0.580/5	8909/-/0.430/4	8886/-/0.680/7	8909/-/0.630/6	8929/-/0.201/3
Weish 23	8344/-/0.000/1	8344/-/0.016/3	8344/-/0.000/1	8344/-/0.850/5	8303/-/0.660/4	8250/-/1.160/7	8344/-/0.890/6	8344/-/0.000/1
Weish24	10220/-/0.000/1	10220/-/0.005/3	10220/-/0.000/1	10220/-/0.160/4	10189/-/1.230/6	10058/-/3.140/7	10215/-/0.860/5	10220/-/0.030/4
Weish 25	9939/-/0.000/1	9939/-/0.113/4	9939/-/0.000/1	9939/-/0.030/3	9885/-/0.940/6	9844/-/1.630/7	9939/-/0.200/5	9939/-/0.000/1
Weish26	9584/-/0.000/1	9584/-/0.000/1	9584/-/0.000/1	9584/-/0.190/4	9575/-/0.690/5	9575/-/1.170/7	9575/-/0.920/6	9578/-/0.066/4
Weish 27	9819/-/0.000/1	9819/-/0.000/1	9819/-/0.000/1	9819/-/0.500/5	9778/-/0.450/4	9589/-/2.530/7	9764/-/0.880/6	9781/-/0.390/4
Weish28	9492/-/0.000/1	9492/-/0.000/1	9492/-/0.000/1	9492/-/0.190/4	9454/-/0.490/5	9400/-/1.700/7	9454/-/0.780/6	9454/-/0.400/5
Weish29	9410/-/0.000/1	9410/-/0.234/3	9410/-/0.000/1	9410/-/0.400/4	9369/-/0.430/5	9369/-/1.750/7	9369/-/0.430/5	9372/-/0.404/5
Weish 30	11191/-/0.000/1	11191/-/0.000/1	11191/-/0.000/1	11187/-/0.080/4	11121/-/1.230/6	11025/-/2.490/7	11169/-/0.610/5	11191/-/0.032/4
cb1-1	24381/24365.200/0.065/3	24381/24367.700/0.055/2	24381/24369.600/0.047/1	24381/24301.000/0.328/4	-/-	24192/24050.000/1.358/5	-/-	-/-
cb1-2	24274/24265.200/0.036/4	24274/24271.800/0.009/2	24274/24267.400/0.027/3	24274/24231.000/0.177/5	-/-	24274/24274.000/0.000/1	-/-	-/-
cb1-3	23538/23527.450/0.100/1	23534/23523.550/0.117/3	23538/23524.850/0.111/2	23551/23521.000/0.127/4	-/-	23538/23520.000/0.132/5	-/-	-/-
cb1-4	23511/23489.000/0.191/1	23534/23485.900/0.204/3	23527/23486.800/0.201/2	23503/23420.000/0.484/4	-/-	23288/23120.000/1.759/5	-/-	-/-
cb1-5	23966/23958.450/0.136/2	23991/23950.350/0.169/3	23991/23959.000/0.133/1	23966/23937.000/0.225/4	-/-	23947/23900.000/0.379/5	-/-	-/-
cb 2-1	59312/59222.250/0.151/3	59243/59241.150/0.119/1	59243/59223.850/0.149/2	59063/58862.000/0.759/4	-/-	58473/58240.000/1.807/5	-/-	-/-
cb 2-2	61454/61410.000/0.101/2	61450/61374.600/0.158/3	61468/61428.350/0.071/1	61295/61051.000/0.685/4	-/-	60692/60390.000/1.760/5	-/-	-/-
cb 2-3	62065/62027.850/0.164/2	62057/61970.450/0.257/3	62057/62031.800/0.158/1	61767/61552.000/0.930/4	-/-	61702/61330.000/1.288/5	-/-	-/-
cb 2-4	59427/59394.050/0.116/3	59429/59407.400/0.094/1	59425/59398.850/0.108/2	59140/58922.000/0.910/4	-/-	58441/58300.000/1.956/5	-/-	-/-
cb 2-5	58950/58930.450/0.035/3	58950/58945.450/0.009/1	58950/58937.650/0.023/2	58605/58390.000/0.952/4	-/-	58082/58300.000/1.104/5	-/-	-/-

cb3-1	120130/120055.850/0.077/2	120116/120083.400/0.054/1	120090/120053.600/0.079/3	119101/118457.000/1.407/5	-/-	119978/119900.000/0.206/4	-/-	-/-
cb3-2	117823/117768.000/0.094/3	117824/117811.750/0.057/1	117814/117779.600/0.084/2	116227/115704.000/1.845/4	-/-	115634/115400.000/2.103/5	-/-	-/-
cb3-3	121121/121057.250/0.061/3	121121/121088.550/0.035/1	121121/121057.450/0.061/2	119990/119468.000/1.373/4	-/-	119156/118900.000/1.842/5	-/-	-/-
cb3-4	120785/120710.800/0.077/3	120767/120731.200/0.060/2	120782/120731.250/0.060/1	119015/118386.000/2.002/5	-/-	119124/118900.000/1.576/4	-/-	-/-
cb3-5	122288/122256.200/0.051/3	122319/122264.200/0.045/1	122280/122257.400/0.050/2	120918/120278.000/1.669/5	-/-	121141/120800.000/1.242/4	-/-	-/-
cb4-1	23064/23048.700/0.066/2	23057/23046.300/0.077/3	23064/23052.150/0.051/1	23055/22914.000/0.650/4	-/-	-/-	-/-	-/-
cb4-2	22801/22762.750/0.168/1	22801/22750.700/0.221/3	22801/22753.100/0.210/2	22743/22629.000/0.754/4	-/-	-/-	-/-	-/-
cb4-3	22131/22097.000/0.154/2	22131/22075.050/0.253/3	22131/22097.050/0.153/1	22131/21908.000/1.008/4	-/-	-/-	-/-	-/-
cb4-4	22772/22726.700/0.199/1	22772/22708.600/0.278/3	22772/22724.900/0.207/2	22717/22528.000/1.071/4	-/-	-/-	-/-	-/-
cb4-5	22751/22635.050/0.510/1	22751/22568.200/0.803/4	22751/22614.150/0.602/2	22751/22603.000/0.651/3	-/-	-/-	-/-	-/-
cb5-1	59181/59057.150/0.219/3	59127/59098.150/0.150/1	59127/59072.400/0.194/2	58903/58477.000/1.200/4	-/-	-/-	-/-	-/-
cb5-2	58651/58559.350/0.377/3	58651/58605.950/0.298/1	58605/58570.550/0.358/2	58346/58098.000/1.162/4	-/-	-/-	-/-	-/-
cb5-3	58034/57944.400/0.263/3	58056/57986.650/0.190/1	58055/57971.900/0.215/2	57674/57417.000/1.170/4	-/-	-/-	-/-	-/-
cb5-4	60888/60831.300/0.277/2	60926/60797.200/0.332/3	60977/60867.500/0.217/1	60505/60282.000/1.177/4	-/-	-/-	-/-	-/-
cb5-5	58060/57964.700/0.219/3	58019/57964.800/0.219/2	58060/57975.900/0.200/1	57468/57220.000/1.501/4	-/-	-/-	-/-	-/-
bb6-1	117639/117566.450/0.216/2	117667/117616.000/0.174/1	117659/117562.750/0.219/3	116015/115379.000/2.073/4	-/-	-/-	-/-	-/-
cb6-2	119087/119029.600/0.184/3	119114/119083.950/0.138/1	119105/119079.950/0.142/2	117778/117102.000/1.800/4	-/-	-/-	-/-	-/-
cb6-3	119085/119013.000/0.169/3	119085/119055.950/0.133/1	119085/119020.850/0.163/2	117345/116842.000/1.991/4	-/-	-/-	-/-	-/-
cb6-4	118706/118632.750/0.165/3	118727/118706.600/0.103/1	118727/118638.800/0.160/2	117281/116446.000/2.005/4	-/-	-/-	-/-	-/-
cb6-5	116423/116302.100/0.196/3	116458/116382.200/0.127/1	116404/116309.200/0.189/2	114909/114183.000/2.014/4	-/-	-/-	-/-	-/-
GK-1	3766/3760.050/0.158/1	3759/3757.050/0.238/3	3766/3759.450/0.174/2	3752/3742.000/0.637/5	3743/3736.000/0.797/6	3698/3678.000/2.337/8	3725/3705.000/1.620/7	3756/3745.200/0.552/4
GK-2	3952/3949.500/0.215/3	3952/3949.650/0.211/2	3952/3950.700/0.184/1	3948/3938.000/0.505/4	3949/3931.000/0.682/6	3885/3871.000/2.198/8	3913/3897.000/1.541/7	3944/3935.800/0.561/5
GK-3	5645/5637.700/0.324/3	5644/5639.550/0.291/2	5646/5640.000/0.283/1	5610/5596.000/1.061/5	5613/5596.000/1.061/5	5561/5519.000/2.422/8	5563/5543.000/1.998/7	5617/5610.300/0.808/4
GK-4	5760/5753.450/0.235/1	5758/5749.400/0.305/3	5759/5751.650/0.266/2	5733/5710.000/0.988/5	5712/5701.000/1.144/6	5651/5638.000/2.237/8	5678/5664.000/1.786/7	5732/5715.200/0.898/4
GK-5	7555/7550.350/0.128/3	7555/7551.900/0.107/2	7555/7552.100/0.104/1	7502/7474.000/1.138/6	7499/7485.000/0.992/5	7365/7353.000/2.738/8	7411/7391.000/2.235/7	7528/7507.600/0.693/4
GK-6	7665/7658.900/0.236/1	7665/7656.300/0.270/3	7663/7656.650/0.265/2	7611/7592.000/1.107/6	7607/7597.000/1.042/5	7522/7505.000/2.240/8	7551/7532.000/1.889/7	7649/7622.900/0.705/4
GK-7	19210/19200.050/0.109/3	19213/19209.400/0.060/1	19213/19205.200/0.082/2	19151/19067.000/0.801/6	19110/19102.000/0.619/5	18738/18668.000/2.877/8	18783/18734.000/2.534/7	19173/19146.100/0.390/4
GK-8	18780/18770.000/0.191/2	18776/18772.250/0.179/1	18782/18762.950/0.229/3	18642/18594.000/1.127/6	18641/18627.000/0.952/5	18385/18361.000/2.366/8	18462/18428.000/2.010/7	18705/18681.400/0.663/4
GK-9	58071/58064.250/0.043/3	58077/58070.450/0.032/1	58075/58068.500/0.035/2	57886/57547.000/0.933/6	57868/57843.000/0.423/5	56746/56619.000/2.531/8	56932/56719.000/2.358/7	57915/57889.600/0.343/4

Max/Mean/Pdev/Rank

-: Non Provied